

Unicode in the Library, Part 1: UTF Transcoding

Document #: P2728R3
Date: 2023-05-04
Project: Programming Language C++
Audience: SG-16 Unicode
LEWG-I
LEWG
Reply-to: Zach Laine
<whatwasthataddress@gmail.com>

Contents

- 1 Changelog
 - 1.1 Changes since R0
 - 1.2 Changes since R1
 - 1.3 Changes since R2
- 2 Motivation
 - 2.1 A note about P1629
- 3 The shortest Unicode primer imaginable
- 4 Use cases
 - 4.1 Case 1: Adapt to an existing range interface taking a different UTF
 - 4.2 Case 2: Adapt to an existing iterator interface taking a different UTF
 - 4.3 Case 3: Transcode data as it is read into a buffer
 - 4.4 Case 4: Print the results of transcoding
- 5 Proposed design
 - 5.1 Dependencies
 - 5.2 Add concepts that describe parameters to transcoding APIs
 - 5.3 Add a standard null-terminated sequence sentinel
 - 5.4 Add constants and utility functions that query the state of UTF sequences (well-formedness, etc.)
 - 5.5 Add the transcoding iterators

- 5.6 Add a transcoding view
 - 5.6.1 Add the view proper
 - 5.6.2 Add `as_utfN` view adaptors
 - 5.6.3 Add `utf_view` specialization of formatter
 - 5.6.4 Add `unpack_iterator_and_sentinel` CPO for iterator “unpacking”
- 5.7 Add a feature test macro
- 5.8 Design notes
- 6 Implementation experience
- 7 References

§ 1 Changelog

§ 1.1 Changes since R0

- When naming code points in interfaces, use `char32_t`.
- When naming code units in interfaces, use `charN_t`.
- Remove each eager algorithm, leaving in its corresponding view.
- Remove all the output iterators.
- Change template parameters to `utfN_view` to the types of the from-range, instead of the types of the transcoding iterators used to implement the view.
- Remove all make-functions.
- Replace the misbegotten `as_utfN()` functions with the `as_utfN` view adaptors that should have been there all along.
- Add missing `transcoding_error_handler` concept.
- Turn `unpack_iterator_and_sentinel` into a CPO.
- Lower the UTF iterator concepts from bidirectional to input.

§ 1.2 Changes since R1

- Reintroduce the transcoding-from-a-buffer example.
- Generalize `null_sentinel_t` to a non-Unicode-specific facility.
- In utility functions that search for ill-formed encoding, take a range argument instead of a pair of iterator arguments.
- Replace `utf{8,16,32}_view` with a single `utf_view`.

§ 1.3 Changes since R2

- Add `noexcept` where appropriate.
- Remove non-essential constants and utility functions, and elaborate on the usage of the ones that remain.
- Note differences from similar elements proposed in [P1629R1].
- Extend the examples slightly.
- Correct an error in the description of the view adaptors' semantics, and provide several examples of their use.

§ 2 Motivation

Unicode is important to many, many users in everyday software. It is not exotic or weird. Well, it's weird, but it's not weird to see it used. C and C++ are the only major production languages with essentially no support for Unicode.

Let's fix.

To fix, first we start with the most basic representations of strings in Unicode: UTF. You might get a UTF string from anywhere; on Windows you often get them from the OS, in UTF-16. In web-adjacent applications, strings are most commonly in UTF-8. In ASCII-only applications, everything is in UTF-8, by its definition as a superset of ASCII.

Often, an application needs to switch between UTFs: 8 -> 16, 32 -> 16, etc. In SG-16 we've taken to calling such UTF-N -> UTF-M operations "transcoding".

I'm proposing interfaces to do transcoding that meet certain design requirements that I think are important; I hope you'll agree:

- Ranges are the future. We should have range-friendly ways of doing transcoding. This includes support for sentinels and lazy views.
- Iterators are the present. We should support generic programming, whether it is done in terms of pointers, a particular iterator, or an iterator type specified as a template parameter.
- Transcoding cannot be a black box; sometimes you need to be able to find where there is a break in the encoding, or to detect whether a sequence has any broken encodings in it. We should provide utility functions that let users investigate these states.
- A null-terminated string should not be treated as a special case. The ubiquity of such strings means that they should be treated as first-class strings.
- It is common to want to view the same text as code points and code units at different times. It is therefore important that transcoding iterators have a convenient way to access the underlying sequence of code units being transcribed.

§ 2.1 A note about P1629

[P1629R1] from JeanHeyd Meneide is a much more ambitious proposal that aims to standardize a general-purpose text encoding conversion mechanism. This proposal is not at odds with P1629; the two proposals have largely orthogonal aims. This proposal only concerns itself with UTF interconversions, which is all that is required for Unicode support. P1629 is concerned with those conversions, plus a lot more. Accepting both proposals would not cause problems; in fact, the APIs proposed here could be used to implement parts of the P1629 design.

There are some differences between the way that the transcode views and iterators from [P1629R1] work and the transcoding view and iterators from this paper work. First, `std::text::transcode_view` has no direct support for null-terminated strings. Second, it does not do the unpacking described in this paper. Third, they are not printable and streamable.

§ 3 The shortest Unicode primer imaginable

There are multiple encoding types defined in Unicode: UTF-8, UTF-16, and UTF-32.

A *code unit* is the lowest-level datum-type in your Unicode data. Examples are a `char` in UTF-8 and a `char32_t` in UTF-32.

A *code point* is a 32-bit integral value that represents a single Unicode value. Examples are U+0041 “A” “LATIN CAPITAL LETTER A” and U+0308 “” “COMBINING DIAERESIS”.

A code point may be consist of multiple code units. For instance, 3 UTF-8 code units in sequence may encode a particular code point.

§ 4 Use cases

§ 4.1 Case 1: Adapt to an existing range interface taking a different UTF

In this case, we have a generic range interface to transcode into, so we use a transcoding view.

```
// A generic function that accepts sequences of UTF-16.
template<std::uc::utf16_range R>
void process_input(R r);
void process_input_again(std::uc::utf_view<std::uc::format::utf16,
std::ranges::ref_view<std::string>> r);

std::u8string input = get_utf8_input();
auto input_utf16 = std::views::all(input) | std::uc::as_utf16;
```

```
process_input(input_utf16);
process_input_again(input_utf16);
```

§ 4.2 Case 2: Adapt to an existing iterator interface taking a different UTF

This time, we have a generic iterator interface we want to transcode into, so we want to use the transcoding iterators.

```
// A generic function that accepts sequences of UTF-16.
template<std::uc::utf16_iter I>
void process_input(I first, I last);

std::u8string input = get_utf8_input();

process_input(std::uc::utf_8_to_16_iterator(input.begin(), input.begin(),
                                           std::uc::utf_8_to_16_iterator(input.begin(), input.end(),
                                           input.end())));

// Even more conveniently:
auto const utf16_view = std::views::all(input) | std::uc::as_utf16;
process_input(utf16_view.begin(), utf16_view.end());
```

§ 4.3 Case 3: Transcode data as it is read into a buffer

Let's say we have a wire-communications layer that knows nothing about the UTFs, and we need to use some of the utility functions to make sure we don't process partially-received UTF-8 sequences.

```
// Using same size to ensure the transcode operation always has room.
char utf8_buf[buf_size];
char utf16_buf[buf_size];

char * read_first = utf8_buf;
while (true) {
    // Reads off a wire; may contain partial UTF-8 sequences at the ends of
    // some reads.
    char * buf_last = read_into_utf8_buffer(read_first, utf8_buf +
                                           buf_size);

    if (buf_last == read_first)
        continue;

    // find the last whole UTF-8 sequence, so we don't feed partial
    // to the algorithm below.
    char * last = buf_last;
    auto const last_lead = std::ranges::find_last_if(
        utf8_buf, buf_last, std::uc::is_lead_code_unit);
    if (!last_lead.empty()) {
        auto const dist_from_end = buf_last - last_lead.begin();
        assert(dist_from_end <= 4);
    }
}
```

```

        if (std::uc::utf8_code_units(*last_lead.begin()) != dist_from_end)
            last = last_lead.begin();
    }

    auto const result = std::ranges::copy(
        std::ranges::subrange(utf8_buf, last) | std::uc::as_utf16,
        utf16_buf);

    // Do something with the resulting UTF-16 buffer contents.
    send_utf16_somewhere(utf16_buf, result.out);

    // Copy partial UTF-8 sequence to start of buffer.
    read_first = std::ranges::copy_backward(last, buf_last, utf8_buf).out;
}

```

§ 4.4 Case 4: Print the results of transcoding

Text processing is pretty useless without I/O. All of the Unicode algorithms operate on code points, and so the output of any of those algorithms will be in code points/UTF-32. It should be easy to print the results to a `std::ostream`, to a `std::wostream` on Windows, or using `std::print`. `utf_view` is therefore printable and streamable.

```

void double_print(char32_t const * str)
{
    auto utf8 = str | std::uc::as_utf8;
    std::print(utf8);
    std::cerr << utf8;
}

```

§ 5 Proposed design

§ 5.1 Dependencies

This proposal depends on the existence of [P2727](#) “`std::iterator_interface`”.

§ 5.2 Add concepts that describe parameters to transcoding APIs

```

namespace std::uc {

    enum class format { utf8 = 1, utf16 = 2, utf32 = 4 };

    template<class T, format F>
    concept code_unit = integral<T> && sizeof(T) == (int)F;

    template<class T>
    concept utf8_code_unit = code_unit<T, format::utf8>;

    template<class T>
    concept utf16_code_unit = code_unit<T, format::utf16>;
}

```

```

template<class T>
    concept utf32_code_unit = code_unit<T, format::utf32>;

template<class T>
    concept utf_code_unit = utf8_code_unit<T> || utf16_code_unit<T> ||
        utf32_code_unit<T>;

template<class T, format F>
    concept code_unit_iter =
        input_iterator<T> && code_unit<iter_value_t<T>, F>;
template<class T, format F>
    concept code_unit_pointer =
        is_pointer_v<T> && code_unit<iter_value_t<T>, F>;
template<class T, format F>
    concept code_unit_range = ranges::input_range<T> &&
        code_unit<ranges::range_value_t<T>, F>;

template<class T>
    concept utf8_iter = code_unit_iter<T, format::utf8>;
template<class T>
    concept utf8_pointer = code_unit_pointer<T, format::utf8>;
template<class T>
    concept utf8_range = code_unit_range<T, format::utf8>;

template<class T>
    concept utf16_iter = code_unit_iter<T, format::utf16>;
template<class T>
    concept utf16_pointer = code_unit_pointer<T, format::utf16>;
template<class T>
    concept utf16_range = code_unit_range<T, format::utf16>;

template<class T>
    concept utf32_iter = code_unit_iter<T, format::utf32>;
template<class T>
    concept utf32_pointer = code_unit_pointer<T, format::utf32>;
template<class T>
    concept utf32_range = code_unit_range<T, format::utf32>;

template<class T>
    concept utf_iter = utf8_iter<T> || utf16_iter<T> || utf32_iter<T>;
template<class T>
    concept utf_pointer = utf8_pointer<T> || utf16_pointer<T> ||
        utf32_pointer<T>;
template<class T>
    concept utf_range = utf8_range<T> || utf16_range<T> || utf32_range<T>;

template<class T>
    concept utf_range_like =
        utf_range<remove_reference_t<T>> ||
        utf_pointer<remove_reference_t<T>>;

template<class T>
    concept utf8_input_range_like =
        (ranges::input_range<remove_reference_t<T>> &&
         utf8_code_unit<iter_value_t<T>>) ||
        utf8_pointer<remove_reference_t<T>>;

```

```

template<class T>
concept utf16_input_range_like =
    (ranges::input_range<remove_reference_t<T>>    &&
     utf16_code_unit<iter_value_t<T>>) ||
    utf16_pointer<remove_reference_t<T>>;
template<class T>
concept utf32_input_range_like =
    (ranges::input_range<remove_reference_t<T>>    &&
     utf32_code_unit<iter_value_t<T>>) ||
    utf32_pointer<remove_reference_t<T>>;

template<class T>
concept utf_input_range_like =
    utf8_input_range_like<T> || utf16_input_range_like<T> ||
    utf32_input_range_like<T>;

template<class T>
concept transcoding_error_handler =
    requires(T t, char const * msg) { { t(msg) } -> code_point; };
}

```

§ 5.3 Add a standard null-terminated sequence sentinel

```

namespace std {
    struct null_sentinel_t {
        constexpr null_sentinel_t base() const noexcept { return {}; }

        template<class T>
        friend constexpr bool operator==(const T* p, null_sentinel_t)
            { return *p == T{}; }
    };

    inline constexpr null_sentinel_t null_sentinel;
}

```

The `base()` member bears explanation. It is there to make iterator/sentinel pairs easy to use in a generic context. Consider a range `r1` of code points delimited by a pair of `utf_8_to_32_iterator<char const *>` transcoding iterators (defined later in this paper). The range of underlying UTF-8 code units is `[r1.begin().base(), r1.end().base())`.

Now consider a range `r2` of code points that is delimited by a `utf_8_to_32_iterator<char const *>` transcoding iterator and a `null_sentinel`. Now our underlying range of UTF-8 is `[r.begin().base(), null_sentinel)`.

Instead of making people writing generic code have to special-case the use of `null_sentinel`, `null_sentinel` has a `base()` member that lets us write `r.end().base()` instead of `null_sentinel`. This means that for either `r` or `r2`, the underlying range of UTF-8 code units is just `[r1.begin().base(), r1.end().base())`.

Note that this is a general-interest utility, and as such, it is in `std`, not `std::uc`.

§ 5.4 Add constants and utility functions that query the state of UTF sequences (well-formedness, etc.)

```
namespace std::uc {
    inline constexpr char32_t replacement_character = 0xfffd;

    // Given the first (and possibly only) code unit of a UTF-8-encoded code
    // point, returns the number of bytes occupied by that code point (in
    // the
    // range [1, 4]). Returns a value < 0 if first_unit is not a valid
    // initial UTF-8 code unit.
    constexpr int utf8_code_units(char8_t first_unit) noexcept;

    // Returns true iff c is a UTF-8 continuation (non-lead) code unit.
    constexpr bool is_continuation(char8_t c) noexcept;

    // Given the first (and possibly only) code unit of a UTF-16-encoded
    // code
    // point, returns the number of code units occupied by that code point
    // (in the range [1, 2]). Returns a value < 0 if first_unit is
    // not a valid initial UTF-16 code unit.
    constexpr int utf16_code_units(char16_t first_unit) noexcept;

    // Returns true iff c is a Unicode low (non-lead) surrogate.
    constexpr bool is_low_surrogate(char32_t c) noexcept;

    // Returns the first code unit in [ranges::begin(r), ranges::end(r))
    // that
    // is not properly UTF-8 encoded, or ranges::begin(r) +
    // ranges::distance(r) if
    // no such code unit is found.
    template<utf8_range R>
    requires ranges::forward_range<R>
    constexpr ranges::borrowed_iterator_t<R> find_invalid_encoding(R &&
        r) noexcept;

    // Returns the first code unit in [ranges::begin(r), ranges::end(r))
    // that
    // is not properly UTF-16 encoded, or ranges::begin(r) +
    // ranges::distance(r) if
    // no such code unit is found.
    template<utf16_range R>
    requires ranges::forward_range<R>
    constexpr ranges::borrowed_iterator_t<R> find_invalid_encoding(R &&
        r) noexcept;

    // Returns true iff r is empty or the initial UTF-8 code units in r form
    // a valid
    // Unicode code point.
    template<utf8_range R>
    requires ranges::forward_range<R>
    constexpr bool starts_encoded(R && r) noexcept;

    // Returns true iff r is empty or the initial UTF-16 code units in r
    // form a valid
    // Unicode code point.
```

```

template<utf16_range R>
    requires ranges::forward_range<R>
    constexpr bool starts_encoded(R && r) noexcept;

// Returns true iff r is empty or the final UTF-8 code units in r form a
valid
// Unicode code point.
template<utf8_range R>
    requires ranges::bidirectional_range<R> && ranges::common_range<R>
    constexpr bool ends_encoded(R && r) noexcept;

// Returns true iff r is empty or the final UTF-16 code units in r form
a valid
// Unicode code point.
template<utf16_range R>
    requires ranges::bidirectional_range<R> && ranges::common_range<R>
    constexpr bool ends_encoded(R && r) noexcept;
}

```

These utility functions are useful for finding encoding breakages in UTF ranges.

utf8_code_units can be used to determine whether a UTF-8 code unit is an initial code unit within a code point sequence, and if so, how many continuation code units are to follow. is_continuation can then be used to verify that the N expected code units in the code point sequence are actually continuation code units. This sort of inquiry is useful in cases like Case 3 example near the top of the paper. utf16_code_units and is_low_surrogate form a similar pair for UTF-16.

The other functions can be used to check if a given range is properly UTF-8 or -16 encoded, either entirely, or at the beginning or end of the range.

§ 5.5 Add the transcoding iterators

I'm using P2727's iterator_interface here for simplicity.

```

namespace std::uc {
    // An error handler type that can be used with the converting iterators;
    // provides the Unicode replacement character on errors.
    struct use_replacement_character {
        constexpr char32_t operator()(const char*) const noexcept { return
            replacement_character; }
    };

    template<class I>
    auto bidirectional_at_most() { // exposition only
        if constexpr (bidirectional_iterator<I>) {
            return bidirectional_iterator_tag{};
        } else if constexpr (forward_iterator<I>) {
            return forward_iterator_tag{};
        } else if constexpr (input_iterator<I>) {
            return input_iterator_tag{};
        }
    }
}

```

```

template<class I>
using bidirectional-at-most-t = decltype(bidirectional-at-most<I>()); //
    exposition only

template<
    utf32_iter I,
    sentinel_for<I> S = I,
    transcoding_error_handler ErrorHandler = use_replacement_character>
struct utf_32_to_8_iterator
    : iterator_interface<utf_32_to_8_iterator<I, S, ErrorHandler>,
        bidirectional-at-most-t<I>, char8_t, char8_t> {
    constexpr utf_32_to_8_iterator();
    explicit constexpr utf_32_to_8_iterator(I first, I it, S last);
    template<class I2, class S2>
        requires convertible_to<I2, I> && convertible_to<S2, S>
        constexpr utf_32_to_8_iterator(
            const utf_32_to_8_iterator<I2, S2, ErrorHandler>& other);

    constexpr I begin() const { return first_; }
    constexpr S end() const { return last_; }

    constexpr char8_t operator*() const { return buf_[index_]; }

    constexpr I base() const { return it_; }

    constexpr utf_32_to_8_iterator& operator++();
    constexpr utf_32_to_8_iterator& operator--();

    template<class I1, class S1, class I2, class S2, class ErrorHandler2>
    friend constexpr bool operator==(
        const utf_32_to_8_iterator<I1, S1, ErrorHandler2>& lhs,
        const utf_32_to_8_iterator<I2, S2, ErrorHandler2>& rhs)
        requires requires { lhs.base() == rhs.base(); }
        { return lhs.base() == rhs.base() && lhs.index_ == rhs.index_; }

    friend constexpr bool operator==(utf_32_to_8_iterator lhs,
        utf_32_to_8_iterator rhs)
        { return lhs.base() == rhs.base() && lhs.index_ == rhs.index_; }

    using base-type = // exposition only
        iterator_interface<utf_32_to_8_iterator<I, S, ErrorHandler>,
            bidirectional-at-most-t<I>,
            char8_t,
            char8_t>;
    using base-type::operator++;
    using base-type::operator--;

private:
    I first_; // exposition only
    I it_; // exposition only
    S last_; // exposition only
    int index_; // exposition only
    array<char8_t, 5> buf_; // exposition only

```

```

    template<utf32_iter I2, sentinel_for<I2> S2, transcoding_error_handler
        ErrorHandler2>
    friend struct utf_32_to_8_iterator;
};

template<class I, class S, class ErrorHandler>
constexpr bool operator==(
    utf_32_to_8_iterator<I, S, ErrorHandler> lhs, S rhs)
    requires requires { lhs.base() == rhs; };

template<
    utf8_iter I,
    sentinel_for<I> S = I,
    transcoding_error_handler ErrorHandler = use_replacement_character>
struct utf_8_to_32_iterator
    : iterator_interface<utf_8_to_32_iterator<I, S, ErrorHandler>,
        bidirectional-at-most-t<I>, char32_t, char32_t> {
    constexpr utf_8_to_32_iterator();
    explicit constexpr utf_8_to_32_iterator(I first, I it, S last);
    template<class I2, class S2>
        requires convertible_to<I2, I> && convertible_to<S2, S>
        constexpr utf_8_to_32_iterator(
            const utf_8_to_32_iterator<I2, S2, ErrorHandler>& other);

    constexpr I begin() const { return first_; }
    constexpr S end() const { return last_; }

    constexpr char32_t operator*() const;

    constexpr I base() const { return it_; }

    constexpr utf_8_to_32_iterator& operator++();
    constexpr utf_8_to_32_iterator& operator--();

    friend constexpr bool operator==(utf_8_to_32_iterator lhs,
        utf_8_to_32_iterator rhs)
    { return lhs.base() == rhs.base(); }

    using base-type = // exposition only
        iterator_interface<utf_8_to_32_iterator<I, S, ErrorHandler>,
            bidirectional-at-most-t<I>,
            char32_t,
            char32_t>;
    using base-type::operator++;
    using base-type::operator--;

private:
    I first_; // exposition only
    I it_; // exposition only
    S last_; // exposition only

    template<utf8_iter I2, sentinel_for<I2> S2, transcoding_error_handler
        ErrorHandler2>
    friend struct utf_8_to_16_iterator;

```

```

    template<utf8_iter I2, sentinel_for<I2> S2, transcoding_error_handler
        ErrorHandler2>
    friend struct utf_8_to_32_iterator;
};

template<class I, class S, class ErrorHandler>
constexpr bool operator==(
    const utf_8_to_32_iterator<I, S, ErrorHandler>& lhs, Sentinel rhs)
    requires requires { lhs.base() == rhs; };

template<class I1, class S1, class I2, class S2, class ErrorHandler>
constexpr bool operator==(
    const utf_8_to_32_iterator<I1, S1, ErrorHandler>& lhs,
    const utf_8_to_32_iterator<I2, S2, ErrorHandler>& rhs)
    requires requires { lhs.base() == rhs.base(); };

template<
    utf32_iter I,
    sentinel_for<I> S = I,
    transcoding_error_handler ErrorHandler = use_replacement_character>
struct utf_32_to_16_iterator
    : iterator_interface<utf_32_to_16_iterator<I, S, ErrorHandler>,
        bidirectional-at-most-t<I>, char16_t, char16_t> {
    constexpr utf_32_to_16_iterator();
    explicit constexpr utf_32_to_16_iterator(I first, I it, S last);
    template<class I2, class S2>
        requires convertible_to<I2, I> && convertible_to<S2, S>
        constexpr utf_32_to_16_iterator(
            const utf_32_to_16_iterator<I2, S2, ErrorHandler>& other);

    constexpr I begin() const { return first_; }
    constexpr S end() const { return last_; }

    constexpr char16_t operator*() const
    { return buf_[index_]; }

    constexpr I base() const { return it_; }

    constexpr utf_32_to_16_iterator& operator++();
    constexpr utf_32_to_16_iterator& operator--();

    template<class I1, class S1, class I2, class S2, class ErrorHandler2>
    friend constexpr bool operator==(
        const utf_32_to_16_iterator<I1, S1, ErrorHandler2>& lhs,
        const utf_32_to_16_iterator<I2, S2, ErrorHandler2>& rhs)
        requires requires { lhs.base() == rhs.base(); }
        { return lhs.base() == rhs.base() && lhs.index_ == rhs.index_; }

    friend constexpr bool operator==(utf_32_to_16_iterator lhs,
        utf_32_to_16_iterator rhs)
        { return lhs.base() == rhs.base() && lhs.index_ == rhs.index_; }

    using base-type = // exposition only
        iterator_interface<utf_32_to_16_iterator<I, S, ErrorHandler>,
            bidirectional-at-most-t<I>,
            char16_t,

```

```

        char16_t>;
using base-type::operator++;
using base-type::operator--;

private:
    I first_;           // exposition only
    I it_;              // exposition only
    S last_;            // exposition only
    int index_;         // exposition only
    array<char16_t, 4> buf_; // exposition only

    template<utf32_iter I2, sentinel_for<I2> S2, transcoding_error_handler
        ErrorHandler2>
    friend struct utf_32_to_16_iterator;
};

template<class I, class S, class ErrorHandler>
constexpr bool operator==(
    const utf_32_to_16_iterator<I, S, ErrorHandler>& lhs, Sentinel rhs)
    requires requires { lhs.base() == rhs; };

template<
    utf16_iter I,
    sentinel_for<I> S = I,
    transcoding_error_handler ErrorHandler = use_replacement_character>
struct utf_16_to_32_iterator
    : iterator_interface<utf_16_to_32_iterator<I, S, ErrorHandler>,
        bidirectional-at-most-t<I>, char32_t, char32_t> {
    constexpr utf_16_to_32_iterator();
    explicit constexpr utf_16_to_32_iterator(I first, I it, S last);
    template<class I2, class S2>
        requires convertible_to<I2, I> && convertible_to<S2, S>
        constexpr utf_16_to_32_iterator(
            const utf_16_to_32_iterator<I2, S2, ErrorHandler>& other);

    constexpr I begin() const { return first_; }
    constexpr S end() const { return last_; }

    constexpr char32_t operator*() const;

    constexpr I base() const { return it_; }

    constexpr utf_16_to_32_iterator& operator++();
    constexpr utf_16_to_32_iterator& operator--();

    friend constexpr bool operator==(utf_16_to_32_iterator lhs,
        utf_16_to_32_iterator rhs)
        { return lhs.base() == rhs.base(); }

    using base-type =           // exposition only
        iterator_interface<utf_16_to_32_iterator<I, S, ErrorHandler>,
            bidirectional-at-most-t<I>,
            char32_t,
            char32_t>;
using base-type::operator++;
using base-type::operator--;

```

```

private:
    I first_;           // exposition only
    I it_;              // exposition only
    S last_;            // exposition only

    template<utf32_iter I2, sentinel_for<I2> S2, transcoding_error_handler
        ErrorHandler2>
    friend struct utf_32_to_16_iterator;

    template<utf16_iter I2, sentinel_for<I2> S2, transcoding_error_handler
        ErrorHandler2>
    friend struct utf_16_to_32_iterator;
};

template<class I, class S, class ErrorHandler>
constexpr bool operator==(
    const utf_16_to_32_iterator<I, S, ErrorHandler>& lhs, Sentinel rhs)
    requires requires { lhs.base() == rhs; };

template<
    class I1, class S1,
    class I2, class S2,
    class ErrorHandler>
constexpr bool operator==(
    const utf_16_to_32_iterator<I1, S1, ErrorHandler>& lhs,
    const utf_16_to_32_iterator<I2, S2, ErrorHandler>& rhs)
    requires requires { lhs.base() == rhs.base(); };

template<
    utf16_iter I,
    sentinel_for<I> S = I,
    transcoding_error_handler ErrorHandler = use_replacement_character>
struct utf_16_to_8_iterator
    : iterator_interface<utf_16_to_8_iterator<I, S, ErrorHandler>,
        bidirectional-at-most-t<I>, char8_t, char8_t> {
    constexpr utf_16_to_8_iterator();
    explicit constexpr utf_16_to_8_iterator(I first, I it, S last);
    template<class I2, class S2>
        requires convertible_to<I2, I> && convertible_to<S2, S>
        constexpr utf_16_to_8_iterator(const utf_16_to_8_iterator<I2, S2>&
            other);

    constexpr I begin() const { return first_; }
    constexpr S end() const { return last_; }

    constexpr char8_t operator*() const { return buf_[index_]; }

    constexpr I base() const { return it_; }

    constexpr utf_16_to_8_iterator& operator++();
    constexpr utf_16_to_8_iterator& operator--();

    template<class I1, class S1, class I2, class S2, class ErrorHandler2>
    friend constexpr bool operator==(
        const utf_16_to_8_iterator<I1, S1, ErrorHandler2>& lhs,

```

```

const utf_16_to_8_iterator<I2, S2, ErrorHandler2>& rhs)
    requires requires { lhs.base() == rhs.base(); }
    { return lhs.base() == rhs.base() && lhs.index_ == rhs.index_; }

    friend constexpr bool operator==(utf_16_to_8_iterator lhs,
utf_16_to_8_iterator rhs)
    { return lhs.base() == rhs.base() && lhs.index_ == rhs.index_; }

using base-type =          // exposition only
    iterator_interface<utf_16_to_8_iterator<I, S, ErrorHandler>,
        bidirectional-at-most-t<I>,
        char8_t,
        char8_t>;
using base-type::operator++;
using base-type::operator--;

private:
    I first_;                // exposition only
    I it_;                   // exposition only
    S last_;                 // exposition only
    int index_;              // exposition only
    array<char8_t, 5> buf_;   // exposition only

    template<utf16_iter I2, sentinel_for<I2> S2, transcoding_error_handler
        ErrorHandler2>
    friend struct utf_16_to_8_iterator;
};

template<class I, class S, class ErrorHandler>
constexpr bool operator==(
    const utf_16_to_8_iterator<I, S, ErrorHandler>& lhs, Sentinel rhs)
    requires requires { lhs.base() == rhs; };

template<class I1, class S1, class I2, class S2, class ErrorHandler>
constexpr bool operator==(
    const utf_16_to_8_iterator<I1, S1, ErrorHandler>& lhs,
    const utf_16_to_8_iterator<I2, S2, ErrorHandler>& rhs)
    requires requires { lhs.base() == rhs.base(); };

template<
    utf8_iter I,
    sentinel_for<I> S = I,
    transcoding_error_handler ErrorHandler = use_replacement_character>
struct utf_8_to_16_iterator
    : iterator_interface<utf_8_to_16_iterator<I, S, ErrorHandler>,
        bidirectional-at-most-t<I>, char16_t, char16_t> {
    constexpr utf_8_to_16_iterator();
    explicit constexpr utf_8_to_16_iterator(I first, I it, S last);
    template<class I2, class S2>
        requires convertible_to<I2, I> && convertible_to<S2, S>
        constexpr utf_8_to_16_iterator(
            const utf_8_to_16_iterator<I2, S2, ErrorHandler>& other);

    constexpr I begin() const { return it_.begin(); }
    constexpr S end() const { return it_.end(); }

```

```

constexpr char16_t operator*() const { return buf_[index_]; }

constexpr I base() const { return it_.base(); }

constexpr utf_8_to_16_iterator& operator++;
constexpr utf_8_to_16_iterator& operator--;

template<class I1, class S1, class I2, class S2, class ErrorHandler2>
friend constexpr bool operator==(
    const utf_8_to_16_iterator<I1, S1, ErrorHandler2>& lhs,
    const utf_8_to_16_iterator<I2, S2, ErrorHandler2>& rhs)
    requires requires { lhs.base() == rhs.base(); }
    { return lhs.base() == rhs.base() && lhs.index_ == rhs.index_; }

    friend constexpr bool operator==(utf_8_to_16_iterator lhs,
    utf_8_to_16_iterator rhs)
    { return lhs.base() == rhs.base() && lhs.index_ == rhs.index_; }

using base-type =                // exposition only
    iterator_interface<utf_8_to_16_iterator<I, S, ErrorHandler>,
        bidirectional-at-most-t<I>,
        char16_t,
        char16_t>;
using base-type::operator++;
using base-type::operator--;

private:
    utf_8_to_32_iterator<I, S> it_; // exposition only
    int index_;                    // exposition only
    array<char16_t, 4> buf_;        // exposition only

    template<utf8_iter I2, sentinel_for<I2> S2, transcoding_error_handler
        ErrorHandler2>
    friend struct utf_8_to_16_iterator;
};

template<class I, class S, class ErrorHandler>
constexpr bool operator==(
    const utf_8_to_16_iterator<I, S, ErrorHandler>& lhs, Sentinel rhs)
    requires requires { lhs.base() == rhs; };
}

```

§ 5.6 Add a transcoding view

§ 5.6.1 Add the view proper

```

namespace std::uc {
    template<class V>
        using utf-view-iter-t = see below; // exposition
        only
    template<class V>
        using utf-view-sent-t = see below; // exposition
        only
    template<format Format, class Unpacked>

```

```

    constexpr auto make-utf-view-iter(Unpacked unpacked); // exposition
    only
template<format Format, class Unpacked>
    constexpr auto make-utf-view-sent(Unpacked unpacked); // exposition
    only

template<format Format, utf_range_like V>
    requires ranges::view<V> || utf_pointer<V>
struct utf_view : ranges::view_interface<utf_view<Format, V>>
{
    using from_iterator = utf-view-iter-t<V>;
    using from_sentinel = utf-view-sent-t<V>;

    using iterator = decltype(make-utf-view-iter<Format>(
        uc::unpack_iterator_and_sentinel(declval<from_iterator>(),
        declval<from_sentinel>())));
    using sentinel = decltype(make-utf-view-iter<Format>(
        uc::unpack_iterator_and_sentinel(declval<from_iterator>(),
        declval<from_sentinel>())));

    constexpr utf_view() {}
    constexpr utf_view(V base);

    constexpr iterator begin() const { return first_; }
    constexpr sentinel end() const { return last_; }

    friend ostream& operator<< (ostream& os, utf_view v);
    friend wostream& operator<< (wostream& os, utf_view v);

private:
    iterator first_;
    [[no_unique_address]] sentinel last_;
};

namespace std::ranges {
    template<uc::format Format, class V>
        inline constexpr bool enable_borrowed_range<uc::utf_view<Format, V>> =
            true;
}

```

utf-view-iter-t evaluates to *v* if *v* is a pointer, and `decltype(std::ranges::begin(std::declval<V>()))` otherwise. *utf-view-sent-t* evaluates to `null_sentinel_t` if *v* is a pointer, and `decltype(std::ranges::end(std::declval<V>()))` otherwise.

make-utf-view-iter makes a transcoding iterator that produces the UTF format from the result of a call to `std::uc::unpack_iterator_and_sentinel()`, and similarly *make-utf-view-sent* makes a sentinel from the result of a call to `std::uc::unpack_iterator_and_sentinel()`.

The `ostream` and `wostream` stream operators transcode the `utf_view` to UTF-8 and UTF-16 respectively (if transcoding is needed), and the `wostream` overload is only defined on Windows.

§ 5.6.2 Add as_utfN view adaptors

Each as_utfN view adaptor adapts a utf_range_like (meaning an range or a null-terminated pointer), and returns a utf_view that may do transcoding (if the inputs are not UTF-N) or the given input (if the inputs are UTF-N).

```
namespace std::uc {  
    inline unspecified as_utf8;  
    inline unspecified as_utf16;  
    inline unspecified as_utf32;  
}
```

Here is some psuedo-wording for as_utfN that hopefully clarifies.

Let E be an expression, and let T be `remove_cvref_t<decltype((E))>`. The expression `as_utfN(E)` is expression-equivalent to:

- If T is a specialization of `empty_view` (`[range.empty.view]`), then `decay-copy(E)`.
- Otherwise, if `is_pointer_v<T>` is true, and T models `code_unit_iter<format::utfN>`, then `ranges::subrange(E, uc::null_sentinel)`.
- Otherwise, if `ranges::iterator_t<T>` models `code_unit_iter<format::utfN>`, then `decay-copy(E)`.
- Otherwise, `utf_view<format::utfN, T>(E)`.

Examples:

```
static_assert(std::is_same_v<  
    decltype(std::views::all(u8"text") | std::uc::as_utf16),  
    std::uc::utf_view<std::uc::format::utf16, std::ranges::ref_view<const  
        char8_t [5]>>>>);  
  
std::u8string str = u8"text";  
  
static_assert(std::is_same_v<  
    decltype(std::views::all(str) | std::uc::as_utf16),  
    std::uc::utf_view<std::uc::format::utf16,  
        std::ranges::ref_view<std::u8string>>>>);  
  
static_assert(std::is_same_v<  
    decltype(str.c_str() | std::uc::as_utf16),  
    std::uc::utf_view<std::uc::format::utf16, const char8_t *>>>);  
  
static_assert(std::is_same_v<  
    decltype(std::ranges::empty_view<int>{} | std::uc::as_utf16),  
    std::ranges::empty_view<int>>>);  
  
std::u16string str2 = u"text";  
  
static_assert(std::is_same_v<
```

```

decltype(std::views::all(str2) | std::uc::as_utf16),
std::ranges::ref_view<std::u16string>>);

static_assert(std::is_same_v<
    decltype(str2.c_str() | std::uc::as_utf16),
    std::ranges::subrange<const char16_t *, std::uc::null_sentinel_t>>);

```

§ 5.6.3 Add utf_view specialization of formatter

These should be added to the list of “the debug-enabled string type specializations” in [format.formatter.spec]. This allows utf_view to be used in std::format() and std::print(). The intention is that the formatter will transcode to UTF-8 if the formatter’s charT is char, or to UTF-16 if the formatter’s charT is wchar_t – if transcoding is necessary at all.

```

template<uc::format Format, class V>
    struct formatter<uc::utf_view<Format, V>, charT>;

```

§ 5.6.4 Add unpack_iterator_and_sentinel CPO for iterator “unpacking”

```

struct no_op_repacker {
    template<class T>
        T operator()(T x) const { return x; }
};

template<class RepackedIterator, class I, class S, class Then>
struct repacker {
    auto operator()(I it) const { return then(RepackedIterator(first, it,
        last)); }

    I first;
    [[no_unique_address]] S last;
    [[no_unique_address]] Then then;
};

template<format FormatTag, utf_iter I, sentinel_for<I> S, class Repack>
struct utf_tagged_range {
    static constexpr format format_tag = FormatTag;

    I first;
    [[no_unique_address]] S last;
    [[no_unique_address]] Repack repack;
};

// CPO equivalent to:
template<utf_iter I, sentinel_for<I> S, class Repack = no_op_repacker>
constexpr auto unpack_iterator_and_sentinel(I first, S last, Repack repack
    = Repack());

```

A simple way to represent a transcoding view is as a pair of transcoding iterators. However, there is a problem with that approach, since a utf_view<format::utf32, utf_8_to_32_iterator<char const *>> would be a range the size of 6 pointers. Worse

yet, a utf_view<format::utf32, utf_8_to_16_iterator<utf_16_to_32_iterator<char const *>>> would be the size of 18 pointers! Further, such a view would do a UTF-8 to UTF-16 to UTF-32 conversion, when it could have done a direct UTF-8 to UTF-32 conversion instead.

To solve these kinds of problems, utf_view unpacks the iterators it is given in the view it adapts, so that only the bottom-most underlying pointer or iterator is stored:

```
std::string str = "some text";

auto to_16_first = std::uc::utf_8_to_16_iterator<std::string::iterator>(
    str.begin(), str.begin(), str.end());
auto to_16_last = std::uc::utf_8_to_16_iterator<std::string::iterator>(
    str.begin(), str.end(), str.end());

auto to_32_first = std::uc::utf_16_to_32_iterator<
    std::uc::utf_8_to_16_iterator<std::string::iterator>
>(to_16_first, to_16_first, to_16_last);
auto to_32_last = std::uc::utf_16_to_32_iterator<
    std::uc::utf_8_to_16_iterator<std::string::iterator>
>(to_16_first, to_16_last, to_16_last);

auto range = std::ranges::subrange(to_32_first, to_32_last) |
    std::uc::as_utf8;

// Poof! The utf_16_to_32 iterators disappeared!
static_assert(std::is_same<std::ranges::iterator_t<decltype(range)>,
    std::string::iterator>::value, "");
```

Each of these views stores only a single iterator and sentinel, so each view is typically the size of two pointers, and possibly smaller if a sentinel is used.

The same unpacking logic is used in the entire proposed API. This allows you to write `r | std::uc::as_utf32` in a generic context, without caring whether `r` is a range of UTF-8, UTF-16, or UTF-32. You do not need to care about whether `r` is a common range or not. You also can ignore whether `r` is comprised of raw pointers, some other kind of iterator, or transcoding iterators. For example, if `r.begin()` is a `utf_32_to_8_iterator`, the resulting view will use `r.begin().base()` for its begin-iterator.

Sometimes, an interface might accept any UTF-N iterator, and then transcode internally to UTF-32:

```
template<input_iterator I, sentinel_for<I> S, output_iterator<char8_t> O>
    requires(utf8_code_unit<iter_value_t<I>> ||
        utf16_code_unit<iter_value_t<I>>)
transcode_result<I, O> transcode_to_utf32(I first, S last, O out);
```

For such interfaces, it can be difficult in the general case to form an iterator of type `I` to return to the user:

```
template<input_iterator I, sentinel_for<I> S, output_iterator<char8_t> O>
    requires(utf8_code_unit<iter_value_t<I>> ||
        utf16_code_unit<iter_value_t<I>>)
```

```

transcode_result<I, 0> transcode_to_utf32(I first, S last, 0 out) {
    // Get the input as UTF-32.
    auto r = uc::utf_view(uc::format::utf32, first, last);

    // Do transcoding.
    auto copy_result = ranges::copy(r, out);

    // Return an in_out_result.
    return result<I, 0>{/* ??? */, copy_result.out};
}

```

What should we write for `/* ??? */`? That is, how do we get back from the UTF-32 iterator `r.begin()` to an `I` iterator? It's harder than it first seems; consider the case where `I` is `std::uc::utf_16_to_32_iterator<std::uc::utf_8_to_16_iterator<std::string::iterator>>`. The solution is for the unpacking algorithm to remember the structure of whatever iterator it unpacks, and then rebuild the structure when returning the result. To demonstrate, here is the implementation of `transcode_to_utf32` from `Boost.Text`:

```

template<std::input_iterator      I,          std::sentinel_for<I>      S,
        std::output_iterator<char32_t> 0>
        requires(utf8_code_unit<std::iter_value_t<I>>      ||
                 utf16_code_unit<std::iter_value_t<I>>))
transcode_result<I, 0> transcode_to_utf32(I first, S last, 0 out)
{
    auto const r = boost::text::unpack_iterator_and_sentinel(first, last);
    auto unpacked = detail::transcode_to_32<false>(<
        detail::tag_t<r.format_tag>, r.first, r.last, -1, out);
    return {r.repack(unpacked.in), unpacked.out};
}

```

If this all sounds way too complicated, it's not that bad at all. Here's the unpacking/repacking implementation from `Boost.Text`: [unpack.hpp](#).

`unpack_iterator_and_sentinel` is a CPO. It is intended to work with UDTs that provide their own unpacking implementation. It returns a `utf_tagged_range`.

§ 5.7 Add a feature test macro

Add the feature test macro `__cpp_lib_unicode_transcoding`.

§ 5.8 Design notes

None of the proposed interfaces is subject to change in future versions of Unicode; each relates to the guaranteed-stable subset. Just sayin'.

None of the proposed interfaces allocates.

The proposed interfaces allow users to choose amongst multiple convenience-vs-compatibility tradeoffs. Explicitly, they are:

- If you need compatibility with existing iterator-based algorithms (such as the standard algorithms), use the transcoding iterators.
- If you want streamability or the convenience of constructing ranges with a single `| as_utfN` adaptor use, use the transcoding views.

All the transcoding iterators allow you access to the underlying iterator via `.base()`, following the convention of the iterator adaptors already in the standard.

The transcoding views are lazy, as you'd expect. They also compose with the standard view adaptors, so just transcoding at most 10 UTF-16 code units out of some UTF can be done with `foo | std::uc::as_utf16 | std::ranges::views::take(10)`.

Error handling is explicitly configurable in the transcoding iterators. This gives complete control to those who want to do something other than the default. The default, according to Unicode, is to produce a replacement character (`0xfffd`) in the output when broken UTF encoding is seen in the input. This is what all these interfaces do, unless you configure one of the iterators as mentioned above.

The production of replacement characters as error-handling strategy is good for memory compactness and safety. It allows us to store all our text as UTF-8 (or, less compactly, as UTF-16), and then process code points as transcoding views. If an error occurs, the transcoding views will simply produce a replacement character; there is no danger of UB.

Code units are just numbers. All of these interfaces treat integral types as code units of various sizes (at least the ones that are 8-, 16-, or 32-bit). Signedness is ignored.

A null-terminated pointer `p` to an 8-, 16-, or 32-bit string of code units is considered the implicit range `[p, null_sentinel)`. This makes user code much more natural; `"foo" | as_utf16`, `"foo"sv | as_utf16`, and `"foo"s | as_utf16` are roughly equivalent (though the iterator type of the resulting view may differ).

Iterators are constructed from more than one underlying iterator. To do iteration in many text-handling contexts, you need to know the beginning and the end of the range you are iterating over, just to be able to do iteration correctly. Note that this is not a safety issue, but a correctness one. For example, say we have a string `s` of UTF-8 code units that we would like to iterate over to produce UTF-32 code points. If the last code unit in `s` is `0xe0`, we should expect two more code units to follow. They are not present, though, because `0xe0` is the last code unit. Now consider how you would implement `operator++()` for an iterator `iter` that transcodes from UTF-8 to UTF-32. If you advance far enough to get the next UTF-32 code point in each call to `operator++()`, you may run off the end of `s` when you find `0xe0` and try to read two more code units. Note that it does not matter that `iter` probably comes from a range with an end-iterator or sentinel as its mate; inside `iter`'s `operator++()` this is no help. `iter` must therefore have the end-iterator or sentinel as a data member. The same logic applies to the other end of the range if `iter` is bidirectional — it must also have the iterator to the start of the underlying range as a data member. This unfortunate reality comes up over and over in the proposed iterators, not just the ones that

are UTF transcoding iterators. This is why iterators in this proposal (and the ones to come) usually consist of three underlying iterators.

§ 6 Implementation experience

All the interfaces proposed here have been implemented, and re-implemented, several times over the last 5 years or so. They are part of a proposed (but not yet accepted!) Boost library, [Boost.Text](#).

The library has hundreds of stars, though I'm not sure how many users that equates to. All of the interfaces proposed here are among the best-exercised in the library. There are comprehensive tests for all the proposed entities, and those entities are used as the foundation upon which all the other library entities are composed.

Though there are a lot of individual entities proposed here, at one time or another I have need each one of them, though maybe not in every UTF-N -> UTF-M permutation. Those transcoding permutations are there mostly for completeness. I have only ever needed UTF-8 <-> UTF->32 in any of my work that uses Unicode. Frequent Windows users will also need to convert to and from UTF-16 sometimes, because that is the UTF that the OS APIs use.

§ 7 References

[P1629R1] JeanHeyd Meneide. 2020-03-02. Transcoding the world - Standard Text Encoding.

<https://wg21.link/p1629r1>